

Universidad Nacional del Nordeste
Facultad de Ciencias Exactas y Naturales y Agrimensura

Proyecto Integrador: Sistema de Administración de Consortio

Licenciatura en Sistemas de Información

Profesora: Lic. Laura Gomez Solis

Integrantes:

- Escobar, Lucas Joel
- Fernandez, Gabriel Elías
- Kern, Nicolás
- Solis Schneeberger, Matias

Corrientes, 2026

Índice

1. Introducción	4
1.1. Introducción	4
1.2. Breve estado del arte	4
1.3. Objetivos	4
1.4. Fundamentación	5
2. Metodología	6
2.1. Ciclo de Vida - Metodología - SCRUM	6
2.1.1. Fase - Inicio	7
2.1.2. Fase - Planificación y Estimación	9
2.1.2.1. Creación de historias de Usuario.	9
2.1.2.2. Estimación de Historias de Usuario.	11
2.1.2.3. Compromiso de Historias de Usuario.	12
2.1.2.4. Identificación de Tareas.	13
2.1.2.5. Estimación de Tareas	19
2.1.2.6. Actualizar el backlog del sprint	23
2.1.3. 2.1.3 - Fase - Implementación	23
2.1.3.1. Crear entregables.	23
2.1.3.2. Realizar el Daily Standup.	23
2.1.3.3. Refinar el backlog priorizado del producto.	23
2.1.4. 2.1.4 - Fase - Revisión y Retrospectiva	24
2.1.4.1. Demostrar y validar el sprint.	24
2.1.4.2. Retrospectiva del sprint.	24
2.1.5. 2.1.5 - Fase - Liberación	24
2.1.5.1. Enviar Entregables.	24
2.1.5.2. Retrospectiva de la liberación.	24
2.2. Entrevista	24
2.3. 2.3 - Especificación de requisitos de Software	26
2.3.1. 2.3.1 - Introducción	26
2.3.1.1. 2.3.1.2 - Alcance	27
2.3.1.2. 2.3.1.3 - Definiciones, acrónimos y abreviaturas	27
2.3.2. 2.3.2 - Descripción General	27
2.3.2.1. 2.3.2.1 - Perspectiva del producto	27
2.3.2.2. 2.3.2.2 - Funcionalidad del Producto	28
2.3.2.3. 2.3.2.3 - Características de los usuarios	28
2.3.2.4. 2.3.2.4 - Restricciones	29
2.3.2.5. 2.3.2.5 - Suposiciones y dependencias	29
2.3.3. 2.3.3 - Requisitos Específicos	29
2.3.3.1. 2.3.3.1 - Requisitos Funcionales	30
2.3.3.1.1. Gestión de Usuarios.	30

2.3.3.1.2.	Gestión de pagos y expensas	30
2.3.3.1.3.	Gestión de espacios comunes.	31
2.3.3.1.4.	Gestión de visitas y paquetería.	31
2.3.3.1.5.	Gestión de comunicación y documentos.	31
2.3.3.1.6.	Gestión de departamentos y contratos	32
2.3.4.	2.3.4 - Requisitos No Funcionales	33
2.3.4.1.	2.3.4.1 - Requisitos del producto	33
2.3.4.2.	2.3.4.2 - Requisitos organizacionales	34
2.3.4.3.	2.3.4.3 - Requisitos externos	34
2.4.	2.4 - Análisis de Riesgos	34
2.4.1.	2.5 - Planificación del Proyecto	37
2.4.1.1.	2.5.1 - Liberación 1: Gestión Financiera Base	37
2.4.1.2.	2.5.2 - Liberación 2: Gestión de Espacios y Usuarios	38
2.4.1.3.	2.5.3 - Liberación 3: Seguridad y Transparencia	38
2.4.1.4.	2.5.4 - Actividades y Ceremonias Ágiles	38
3.	Herramientas y/o lenguajes de programación	39

Índice de figuras

1.	Lista de BackLog con las épicas.	7
2.	Historia de Usuario 1: Registro del Edificio	9
3.	Historia de Usuario 2: Configuración de Departamentos/Alquileres	10
4.	Mapeo de Historias de Usuario.	11
5.	Simulación de Estimación por afinidad en Trello.	12
6.	Tabla de identificación de Tareas	19
7.	Tabla de estimación de tareas.	23
8.	Tabla de definiciones	27
9.	Tabla de características de usuarios.	28
10.	Tabla de Requisitos: Gestión de usuarios	30
11.	Tabla de requisitos: Gestión de pagos y expensas	30
12.	Tabla de requisitos: Gestión de espacios comunes.	31
13.	Tabla de requisitos: Gestión de visitas y paquetería.	31
14.	Tabla de requisitos: Gestión de comunicación y documentos.	31
15.	Tabla de requisitos: Gestión de comunicación y contratos.	32
16.	Tabla de requisitos: Requisitos del producto	33
17.	Tabla de requisitos: Requisitos organizacionales	34
18.	Tabla de requisitos: Requisitos Externos	34
19.	Tabla de los riesgos que se pensaron	35
20.	Tabla con las probabilidades y seriedad de los riesgos	36
21.	Estrategias para cada riesgo	37

1. Introducción

1.1. Introducción

La administración de consorcios de propiedad horizontal es una tarea compleja que requiere un manejo eficiente de recursos financieros, mantenimiento edilicio y mediación constante entre los residentes. Tradicionalmente, esta labor se ha sostenido mediante métodos manuales, uso de planillas de cálculo genéricas y comunicaciones en papel. El presente proyecto detalla el desarrollo de “CoConsortio”, un sistema web orientado a digitalizar, centralizar y optimizar la gestión integral de edificios, brindando herramientas automatizadas básicas tanto para el administrador como para los propietarios e inquilinos, buscando mejorar la transparencia y la convivencia de forma accesible.

1.2. Breve estado del arte

En la actualidad, la digitalización de la administración de propiedades ha cobrado impulso. Existen en el mercado plataformas comerciales de gran envergadura orientadas a corporaciones inmobiliarias. Sin embargo, estas herramientas de nivel empresarial suelen resultar económicamente inaccesibles para edificios pequeños o medianos, y presentan curvas de aprendizaje muy pronunciadas. En contraste, gran parte de las administraciones locales continúan operando de manera analógica, dependiendo de cuadernos físicos para el control de la guardia o de la distribución de expensas impresas. Esto evidencia una vacante para sistemas más ligeros, enfocados en resolver exclusivamente los problemas críticos diarios. CoConsortio se posiciona en este segmento como una alternativa ágil, sirviendo como una prueba de concepto tecnológica ideal para aquellos consorcios que buscan dar su primer paso hacia la digitalización sin enfrentarse a sistemas sobredimensionados.

1.3. Objetivos

Objetivo General: Desarrollar e implementar un sistema de información web (CoConsortio) para la administración integral de consorcios, que optimice los procesos financieros, operativos y comunicacionales entre la administración, el personal de seguridad y los residentes.

Objetivos Específicos:

- Automatizar el cálculo y prorrateo de liquidaciones mensuales de expensas, reduciendo el margen de error del cálculo manual.
- Implementar una integración inicial con pasarelas de pago (MercadoPago) que permita a los residentes abonar sus deudas de forma electrónica.
- Digitalizar la gestión de espacios comunes mediante un calendario interactivo para evitar conflictos y superposiciones de reservas.
- Centralizar la comunicación institucional mediante un muro de anuncios y un repositorio para la documentación del consorcio.

- Proveer una interfaz de control básica para el personal de seguridad, orientada al registro de visitas y paquetería.

1.4. Fundamentación

El desarrollo de este software surge como respuesta a las problemáticas detectadas en la gestión cotidiana de los edificios, representando a su vez una oportunidad invaluable para aplicar y consolidar los principios de la ingeniería de software en un entorno real. La dependencia de procesos manuales genera demoras en la recaudación y falta de transparencia. Se vuelve imperativo proponer soluciones tecnológicas que reemplacen prácticas obsoletas, como los cuadernos físicos de guardia, que son vulnerables a pérdidas y alteraciones. CoConsortio se justifica técnica y académicamente al plantear una herramienta acotada pero robusta, construida bajo estándares modernos de desarrollo web, que moderniza el flujo de trabajo del administrador y empodera al vecino. Al tratarse de un proyecto gestado en el ámbito universitario, su diseño prioriza el código limpio, la escalabilidad y la correcta aplicación de metodologías ágiles por sobre la saturación de funcionalidades.

2. Metodología

2.1. Ciclo de Vida - Metodología - SCRUM

Scrum es una metodología ágil. Contiene un marco de trabajo adaptable, iterativo, rápido, flexible y eficaz, diseñado para ofrecer un valor considerable de forma rápida a lo largo del proyecto. Para cada proyecto que se aplique esta metodología se debe utilizar los principios scrum que son sus lineamientos básicos para la aplicación de su marco de trabajo y sus aspectos. A continuación, se mencionan las características de los principios y los aspectos del mismo.

Principios de Scrum

1. Control del proceso empírico (Empirical Process Control)
2. Autoorganización (Self-organization)
3. Colaboración (Collaboration)
4. Priorización basada en valor (Value-based Prioritization)
5. Time-boxing
6. Desarrollo iterativo (Iterative Development)

Aspectos de Scrum

1. **Organización:** Entiende los roles y responsabilidades definidos del proyecto con fines de asegurar la implementación exitosa de Scrum. Los roles de Scrum se dividen en 2 categorías: roles centrales (**Product Owner, Scrum Master, Equipo Scrum**) y roles no centrales (Interesados del negocio, clientes, usuarios y patrocinadores/sponsors).
2. **Justificación del negocio:** Se lleva a cabo una evaluación adecuada del negocio antes de iniciar el proyecto.
3. **Calidad:** Es la capacidad con la que cuenta el producto o los entregables para cumplir los con los criterios de aceptación y de alcanzar el valor de negocio esperado por el cliente.
4. **Cambio:** Los miembros del equipo del proyecto deben entender que los procesos de desarrollo están diseñados para el cambio.
5. **Riesgo:** Son una serie de eventos inciertos que pueden afectar los objetivos de un proyecto y pueden contribuir a su éxito o fracaso. Los riesgos que puedan tener impacto positivo en el proyecto, son considerados como oportunidades, mientras los riesgos que puedan tener impacto negativo en el proyecto se les conoce como amenazas.

2.1.1. Fase - Inicio

Visión del proyecto.

Para administradores, vecinos y propietarios del edificio que necesiten resolver la desorganización y demoras en las gestiones diarias, nuestro **Sistema de Administración de consorcio** es una plataforma digital centralizada que automatiza liquidación de expensas, facilita la gestión de vecinos, implementa seguridad tanto dentro del edificio como fuera del mismo y agiliza la comunicación entre Inquilinos/propietarios del edificio. A diferencia de métodos manuales como Whatsapp o papel, en **nuestro sistema se garantiza una transparencia, comunicación rápida and efectiva con un control eficiente** de los recursos del edificio. Hemos identificado a **Fernández Gabriel Elías** como **Product Owner** por la visión del proyecto que proporcionó.

Identificación del Scrum Master.

Bajo los criterios de: **buen liderazgo, asignación de trabajo para los miembros dependiendo de sus habilidades en programación y actitud ante las adversidades**, identificamos a **Matías Solis Schneeberger** como nuestro **Scrum Master**, que nos guiará en este proyecto.

Formación Equipo Scrum.

Nuestro **Product Owner** con la colaboración del **Scrum Master**, ha seleccionado a **Escobar Lucas Joel y Nicolás Kern**, como miembros del equipo Scrum.

Desarrollo de épicas y creación del backlog priorizado del Producto.

Con la declaración de la visión del proyecto hemos podido lograr **desarrollar las épicas esenciales** para nuestro sistema. A continuación se muestran las épicas desarrolladas junto al **Backlog priorizando el producto**.

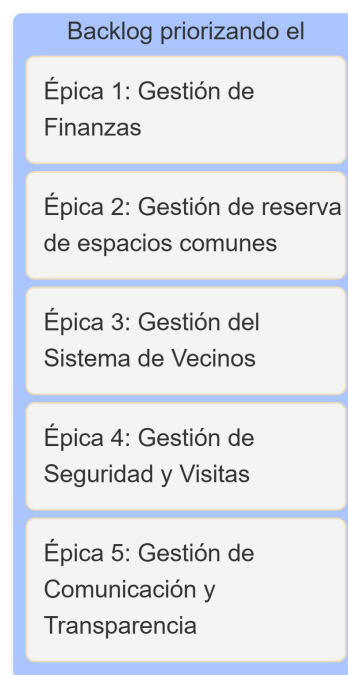


Figura 1: Lista de BackLog con las épicas.

Planificación de la liberación

Tras el análisis del *Product Backlog*, el Equipo Scrum ha priorizado las entregas basándose en el valor de negocio. Se determinó que el núcleo del sistema es el módulo contable, por lo que la Épica 1 (Gestión de Finanzas) conformará la primera liberación. Las entregas subsecuentes abordarán la gestión de reservas, usuarios, seguridad y comunicación.

Liberación 1: Gestión Financiera Base (Duración: 2 Sprints de 2 semanas) Para esta primera entrega, aplicamos una estricta priorización, seleccionando únicamente las Historias de Usuario (HU) críticas para el funcionamiento empírico del sistema, divididas en tres ejes:

- **Estructura Base:** HU1 (Registro del Edificio) y HU2 (Configuración de Departamentos/Alquileres). Son dependencias absolutas; sin la configuración inicial, el sistema carece de viabilidad.
- **Motor de Expenses (Administrador):** HU3 (Ingreso de Gastos), HU5 (Generar liquidación mensual) y HU6 (Registro manual de pagos). Permiten el registro de facturas, la generación de totales y el control de transferencias.
- **Visibilidad (Inquilino/Propietario):** HU11 (Consultar mi estado de deudas), HU15 (Consultar gastos de expensa mensual) y HU13 (Pagar de deudas Online). Garantizan que el usuario final tenga claridad inmediata sobre su estado de cuenta.

Las funcionalidades secundarias de la Épica 1 (HU7 - Cálculo automático de intereses, HU10 - Notificaciones automáticas, HU14 - Subir comprobante y HU16 - Confirmar pago por transferencia) permanecerán en el *Product Backlog* para iteraciones futuras, mitigando así los riesgos técnicos de esta primera entrega.

Liberación 2: Gestión de Espacios y Usuarios (Duración: 2 Sprints de 2 semanas) Esta liberación integrará las funcionalidades esenciales de dos grandes módulos para expandir el valor entregado al consorcio:

- **Épica 2 (Reserva de espacios comunes):** Se implementarán la HU17 (Ver disponibilidad), HU18 (Solicitar reserva) y HU20 (Cancelar reserva), dejando el módulo operativo para los residentes.
- **Épica 3 (Sistema de Inquilinos/Propietarios):** Se abordarán la HU22 (Alta de usuario), HU23 (Actualización de datos) y HU24 (Baja o desvinculación). Las tareas de gestión administrativa profunda (HU19, HU21 y HU25) se postergan para evitar comprometer los plazos y la estabilidad del incremento.

Liberación 3: Seguridad y Transparencia (Duración: 2 Sprints de 2 semanas) Dado que la complejidad técnica y el esfuerzo estimado de las épicas restantes es menor, esta entrega abarca la totalidad de las funcionalidades planificadas para los módulos 4 y 5, sin representar un riesgo para el cronograma del proyecto:

- **Épica 4 (Seguridad y Visitas):** HU26 (Ingreso de visita), HU27 (Recepción de paquetería) y HU28 (Confirmar retiro).
- **Épica 5 (Comunicación):** HU29 (Publicar comunicado), HU30 (Consultar documentos) y HU31 (Enviar consulta a administración).

2.1.2. Fase - Planificación y Estimación

2.1.2.1. Creación de historias de Usuario.

Durante la fase de Planificación y Estimación, el equipo procedió a desglosar las Épicas en requerimientos granulares. Se utilizó el formato estándar de roles y la técnica de comportamiento (Dado/Cuando/Entonces) para los criterios de aceptación, asegurando que cada funcionalidad aporte valor al consorcio. A continuación historias críticas detalladas a modo de ejemplo.

Historia de Usuario 1: Registro del Edificio
Descripción: Como administrador Quiero registrar un nuevo edificio en el sistema definiendo su información general y el desglose de unidades funcionales Para crear el marco estructural necesario sobre el cual se basará toda la gestión administrativa.
Criterios de Aceptación: • Criterio 1: Dado que el administrador está en el formulario de “Alta de Edificio”. Cuando ingresa el nombre, número de departamentos/lotes/pisos y presiona “guardar”. Entonces el sistema crea la entidad del edificio y habilita la configuración de sus departamentos.

Figura 2: Historia de Usuario 1: Registro del Edificio

Historia de Usuario 2: Configuración de Departamentos/Alquileres
Descripción: Como administrador Quiero asignar individualmente a cada unidad funcional su valor de alquiler y su coeficiente de copropiedad Para habilitar la automatización de cobros personalizados y el cálculo preciso de deudas según la situación técnica de cada departamento.
Criterios de Aceptación: Criterio 1: Dado que el edificio ya fue creado y el administrador se encuentra en la ventana de configurar edificio. Cuando el administrador asigna el valor de alquiler y el coeficiente de expensas a cada unidad. Entonces el sistema valida que la suma de todos los coeficientes sea exactamente 100 % antes de permitir el guardado. Criterio 2: Dado que el edificio ya fue creado y el administrador se encuentra en la ventana de configurar edificio. Cuando el administrador asigna el valor de alquiler y el coeficiente de expensas es incorrecto. Entonces el sistema valida que la suma de todos los coeficientes sea exactamente 100 %; al no cumplirse esta condición, no permite el guardado y muestra el mensaje “Datos de coeficientes de expensas incorrectos”.

Figura 3: Historia de Usuario 2: Configuración de Departamentos/Alquileres

Estas historias de usuario corresponden al módulo de Finanzas (Épica 1). Si bien se han detallado las historias críticas a modo de ejemplo, el mapeo exhaustivo del resto de las funcionalidades se encuentra cargado y gestionado en nuestro Sprint backlog en Trello. A continuación imagen del mapeo exhaustivo.

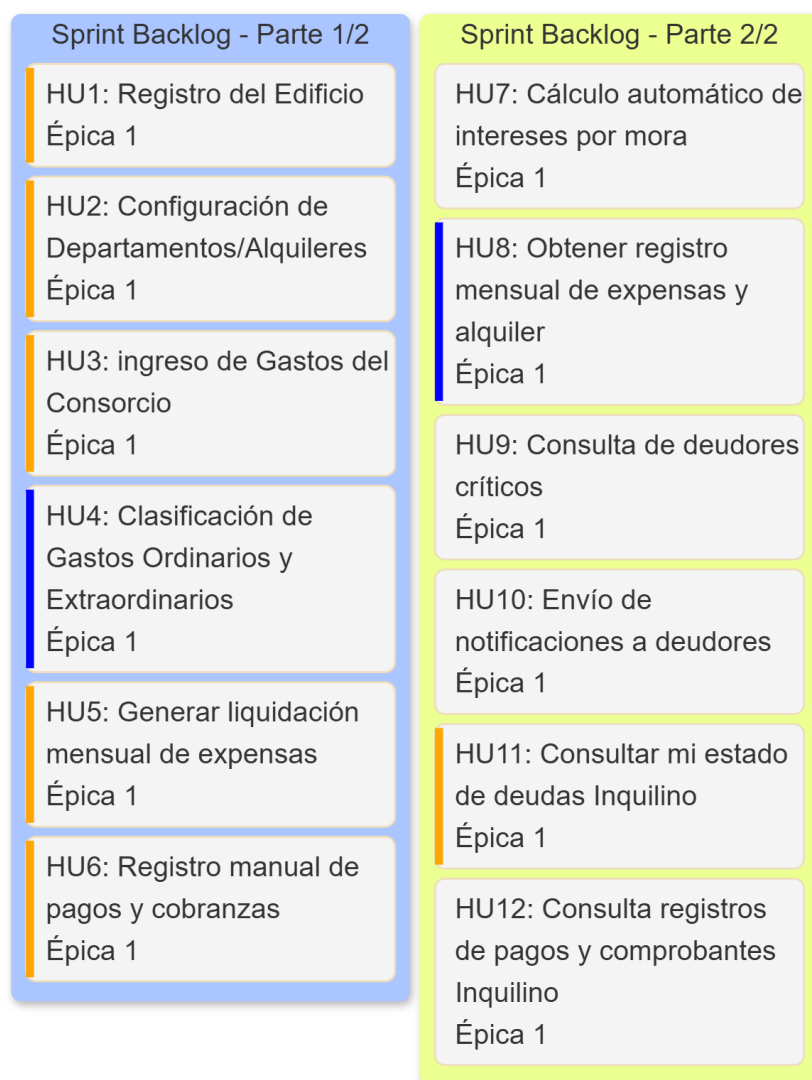


Figura 4: Mapeo de Historias de Usuario.

2.1.2.2. Estimación de Historias de Usuario.

Se aplicó la técnica de **Estimación por afinidad** (Affinity Estimation), categorizando el esfuerzo y complejidad técnica del Product Backlog en tamaños relativos (Pequeño, Mediano, Grande). Esto permite al equipo alinear expectativas rápidamente, detectando que los módulos contables de prorrateo de expensas e integración de pagos representan las historias de mayor tamaño y riesgo técnico (Grande), mientras que los módulos de comunicación estimaron un esfuerzo menor (Pequeño).

Pequeño - Poco esfuerzo requerido	Medio - Esfuerzo moderado requerido	Grande - Mucho esfuerzo requerido
HU9: Consulta de deudores críticos Épica 1	HU3: Ingreso de Gastos del Consorcio Épica 1	HU5: Generar liquidación mensual de expensas Épica 1
HU23: Actualización de datos de contacto Épica 3	HU6: Registro manual de pagos y cobranzas Épica 1	HU1: Registro del Edificio Épica 1
HU17: Ver disponibilidad de los espacios comunes Épica 2	HU26: Registrar ingreso de visita Épica 4	HU2: Configuración de Departamentos/Alquileres Épica 1
HU18: Solicitar reserva de los espacios comunes Épica 2	HU27: Registrar recepción de paquetería Épica 4	HU4: Clasificación de Gastos Ordinarios y Extraordinarios Épica 1
HU19: Gestionar solicitudes de reserva Épica 2	HU22: Alta de un nuevo vecino Épica 3	HU7: Cálculo automático de intereses por mora Épica 1

Figura 5: Simulación de Estimación por afinidad en Trello.

2.1.2.3. Compromiso de Historias de Usuario.

Una vez priorizado el producto backlog y estimado el esfuerzo mediante la técnica de afinidad, el Equipo Scrum llevó a cabo la reunión de planificación del Sprint. El objetivo de la reunión fue definir el **Sprint Backlog** para el primer ciclo de desarrollo (Sprint 1), el cual cuenta con un Time-boxing de dos semanas.

Durante este proceso, el Equipo de Desarrollo evaluó su capacidad operativa y procedió a mover las historias de usuario de mayor prioridad desde la cima del Product Backlog hacia el Sprint Backlog.

Para el **Sprint 1 de la Liberación 1**, el equipo se ha comprometido formalmente a implementar las Historias de Usuario HU1 - Registro del Edificio, HU2 - Configuración de Departamentos/Alquileres, HU5 - Generar liquidación mensual de expensas, HU3 - ingreso de Gastos del Consorcio y HU6: Registro manual de pagos y cobranzas. Al tratarse de requerimientos de tamaño “Grande” y “Mediano” respectivamente, se determinó que estas historias cubren la capacidad de trabajo del equipo para esta primera iteración.

Para el **Sprint 2 de la Liberación 1**, el equipo se ha comprometido formalmente a implementar las Historias de Usuario, HU11: Consultar mi estado de deudas (Inquilino), HU15: Consultar gastos de expensa mensual (Inquilino) y HU13 - Pagar de deudas Online (Inquilino). Al tratarse de requerimientos de tamaños “Mediano” y “Pequeño”, se determinó que estas historias cubren la capacidad de trabajo del equipo en esta segunda iteración.

Para el **Sprint 1 de la Liberación 2**, el equipo se ha comprometido formalmente a implementar las Historias de Usuario HU17 - Ver disponibilidad de los espacios comunes, HU18 - Solicitar reserva de los espacios comunes y HU20 - Cancelar reserva del espacio común. Se determinó

que estas historias cubren la capacidad de trabajo del equipo para esta primera iteración.

Para el **Sprint 2 de la Liberación 2**, el equipo se ha comprometido formalmente a implementar Historias de Usuario HU22 - Alta de un nuevo vecino, HU23 - Actualización de datos de contacto y HU24 - Baja o desvinculación de un inquilino. Se determinó que estas historias cubren la capacidad de trabajo del equipo para esta segunda iteración.

Para el **Sprint 1 de la Liberación 3**, el equipo se ha comprometido formalmente a implementar Historias de Usuario HU26 - Registrar ingreso de visita, HU27 - Registrar recepción de paquetería y HU28 - Confirmar retiro de paquete. Se determinó que estas tareas son las que cubren la capacidad de trabajo del equipo para la primera iteración.

Para el **Sprint 2 de la Liberación 3**, el equipo se ha comprometido formalmente a implementar Historias de Usuario HU29 - Publicar un comunicado general, HU30 - Consultar documentos del consorcio y HU31 - Enviar consulta directa a la administración. Se determinó que estas tareas son las que cubren la capacidad de trabajo del equipo para la segunda iteración.

2.1.2.4. Identificación de Tareas.

Historias de Usuario	Tareas técnicas identificadas
HU1 - Registro del Edificio	1. Diseñar tabla <i>edificios</i> en la base de datos. 2. Crear Modelo y Controlador en Laravel. 3. Desarrollar formulario de Registro (HTML/Blade). 4. Implementar validaciones de campos (Nombre, CUIT, Dirección). 5. Realizar pruebas de persistencia de datos.
HU2: Configuración de Departamentos/Alquileres	1. Diseñar tabla unidades <i>vinculada a edificios</i> en la base de datos. 2. Crear lógica de asignación de coeficientes (porcentaje de las expensas). 3. Desarrollar vista de listado de departamentos.
HU3: Ingreso de Gastos del Consorcio.	1. Diseñar tabla <i>gastos_consortio</i>. 2. Crear formulario de carga de facturas (Monto, Fecha, Concepto) y sección de carga de archivos. 3. Implementar lógica de suma automática de gastos del mes.

Historias de Usuario	Tareas técnicas identificadas
HU5 - Generar liquidación mensual de expensas	1. Diseñar tabla <i>liquidaciones</i> en la base de datos. 2. Crear vista (FrontEnd): maquetar la pantalla del panel de administración que contenga el botón “Cerrar mes” y la ventana emergente de confirmación. 3. Desarrollar lógica de sumatoria (BackEnd): programar en el controlador la consulta que sume todos los gastos cargados en el mes actual. 4. Desarrollar algoritmo de prorratio: Crear la función que tome el total de gastos y lo multiplique por el coeficiente (porcentaje) de cada unidad funcional para sacar el individual. 5. Generar recibos digitales: maquetar la vista o el PDF del comprobante que verá el inquilino con su monto detallado. 6. Implementar validación de seguridad: programar un bloqueo para que el administrador no pueda hacer clic en “Cerrar mes” dos veces para el mismo período (evitar duplicar deudas).
HU6 - Registro manual de pagos y cobranzas	1. Diseñar tabla <i>pagos</i> en la base de datos. 2. Crear formulario de cobro (FrontEnd/Blade): maquetar la pantalla para el administrador (menú desplegable de departamento y campo de monto). 3. Desarrollar la lógica de guardado (Backend/Controlador): Escribir la función que guarde los datos del formulario en la tabla <i>pagos</i>. 4. Actualizar el pago de la deuda (BackEnd): programar lógica que busque la liquidación de ese mes y cambie el estado de “Pendiente” a “Pagado”. 5. Generar comprobante Interno: Maquetar una vista sencilla que muestre un resumen del pago exitoso.

Historias de Usuario	Tareas técnicas identificadas
HU11 - Consultar mi estado de deudas (Inquilino)	1. Desarrollar consulta a la base de datos (BackEnd/Controlador): buscar en la tabla de <i>liquidaciones</i> el registro actual de la unidad funcional. 2. Crear vista principal (FrontEnd/Blade): maquetar la pantalla "Mis deudas" en el panel del inquilino. 3. Programar lógica condicional en la Vista: evaluar si el estado de la liquidación es "Pendiente" o "Pagado". 4. Maquetar escenario "Con Deuda" (Frontend): diseñar la tarjeta de alerta con monto y vencimiento. 5. Maquetar escenario "Sin Deuda" (FrontEnd): diseñar el mensaje de éxito ("Estás al día").
HU15 - Consultar gastos de expensa mensual (Inquilino)	1. Crear botón de acción (FrontEnd): agregar el enlace "ver detalle de gastos" en la liquidación del inquilino. 2. Desarrollar consulta relacional (BackEnd): buscar en la tabla <i>gastos_consortio</i> las facturas correspondientes al mes/año. 3. Maquetar vista de detalle (FrontEnd): diseñar una ventana emergente o tabla desplegable. 4. Formatear la salida de datos: configurar columnas claras (Fecha, concepto, comprobante, monto) y sumatoria coincidente.
HU13 - Pagar de deudas Online (Inquilino)	1. Investigar e integrar API de pagos: Configurar credenciales (Tokens/Keys) en el archivo '.env'. 2. Crear botón de pago (Frontend/Blade): Implementar el botón "Pagar ahora" capturando el monto adeudado. 3. Desarrollar Controlador de Integración (Backend): Enviar datos de la deuda a la API y generar el enlace de pago seguro. 4. Implementar Webhook para confirmación (Backend): Programar ruta para recibir notificaciones de la pasarela. 5. Programar lógica de éxito: Actualizar estado a "Pagado" al recibir estado "Aprobado". 6. Programar lógica de fallo: Mantener estado "Pendiente" y mostrar error en caso de rechazo.

Historias de Usuario	Tareas técnicas identificadas
HU18 - Solicitar reserva de los espacios comunes	1. Diseñar tabla en base de datos (reservas): Crear la migración con campos requeridos. 2. Crear formulario de solicitud (Frontend/Blade): Maquetar formulario con selector de fecha, invitados y reglamento. 3. Desarrollar lógica de validación (Backend/Controlador): Validar fecha a futuro, checkbox y límite de invitados. 4. Programar lógica de guardado (Backend): Persistir la reserva con estado inicial "Pendiente". 5. Implementar disparador de notificación (Backend): Generar alerta interna o enviar correo al Administrador al guardar.
HU20 - Cancelar reserva del espacio común	1. Crear interfaz de cancelación (Frontend/Blade): Agregar botón "Cancelar Reserva" en reservas activas. 2. Desarrollar lógica de cálculo de tiempo (Backend/Controlador): Calcular diferencia en horas respecto a la reserva. 3. Implementar validación condicional (Backend): Evaluar si la diferencia es mayor a 48 horas. 4. Programar escenario de éxito (Criterio 1): Actualizar estado a "Cancelada" y notificar al administrador. 5. Programar escenario de rechazo (Criterio 2): Abortar transacción y retornar error en vista. 6. Maquetar alertas de retroalimentación (Frontend): Diseñar notificaciones visuales para éxito o plazo expirado.
HU17 - Ver disponibilidad de los espacios comunes	1. Integrar componente de calendario (Frontend): Instalar y configurar *FullCalendar.js*. 2. Desarrollar consulta de rango de fechas (Backend): Buscar reservas del mes/año actual en la base de datos. 3. Mapear estados de disponibilidad (Lógica de Interfaz): Asignar clases CSS verde (disponible) o rojo (reservado/pendiente). 4. Programar enrutamiento: Redirigir al clic en verde a "Nueva Reserva" con fecha pre-cargada. 5. Bloquear interacción: Deshabilitar clics en rojo y días pasados.

Historias de Usuario	Tareas técnicas identificadas
HU22 - Alta de un nuevo vecino	1. Diseñar estructura de usuarios (Base de Datos): UNIQUE en 'dni' y 'email' en la tabla 'users'. 2. Crear formulario de Alta (Frontend/Blade): Campos de Nombre, DNI, Email y selector de unidad funcional. 3. Desarrollar lógica de validación (Backend/Controlador): Validar en servidor que DNI y email sean únicos. 4. Programar generación de credenciales (Backend): Generar contraseña aleatoria y encriptarla al crear el perfil. 5. Implementar servicio de mensajería (Backend/Mailable): Enviar correo de bienvenida con credenciales al nuevo inquilino.
HU24 - Baja o desvinculación de un inquilino	1. Maquetar interfaz de desvinculación (Frontend/Blade): Botón "Dar de baja" y modal de confirmación. 2. Desarrollar lógica de Borrado Lógico (Backend): Cambiar estado a "Inactivo" preservando historial. 3. Actualizar estado de la Unidad Funcional (Backend): Desvincular usuario del departamento y marcarlo "Desocupada". 4. Implementar revocación de sesión activa (Seguridad): Middleware para expulsar al usuario si está logueado.
HU23 - Actualización de datos de contacto	1. Crear vista "Mi Perfil" (Frontend/Blade): Formulario pre-cargado con información actual. 2. Desarrollar enrutamiento protegido (Backend): Proteger la ruta para usuarios logueados. 3. Programar validación y sanitización (Backend/Controlador): Validar formato de email y caracteres del teléfono. 4. Ejecutar actualización segura (Backend): Actualizar únicamente teléfono y email del usuario logueado. 5. Implementar retroalimentación visual (Frontend): Retornar a la vista con mensaje de éxito (Toast/Alert).

Historias de Usuario	Tareas técnicas identificadas
HU26 - Registrar ingreso de visita	1. Diseñar tabla de auditoría (visitas): Crear migración con campos necesarios. 2. Crear vista para el rol Guardia (Frontend/Blade): Maquetar panel rápido “Libro de Visitas”. 3. Desarrollar lógica de persistencia (Backend/Controlador): Registrar datos del formulario de visitas. 4. Implementar sellado de tiempo automático: Capturar hora de ingreso del servidor. 5. Configurar estado inicial: Guardar con estado por defecto “En el edificio”. 6. Maquetar tabla de visitas activas (Frontend): Lista de visitas en tiempo real.
HU27 - Registrar recepción de paquetería	1. Diseñar tabla de inventario (paquetes): Crear migración con campos y estado “En guardia”. 2. Crear formulario de recepción (Frontend/Blade): Campos de remitente y selector de departamento. 3. Desarrollar Controlador de registro (Backend): Validar e ingresar paquete asociado a la unidad. 4. Desarrollar lógica de notificación asíncrona (Backend): Evento y Listener para notificar al inquilino por correo/alerta.
HU28 - Confirmar retiro de paquete	1. Modificar vista de inventario (Frontend/Blade): Agregar botón “Marcar como Entregado”. 2. Desarrollar ruta y Controlador (Backend): Crear método que reciba el ID del paquete. 3. Implementar lógica de actualización y auditoría (Backend): Cambiar estado a “Entregado” y registrar hora. 4. Ajustar consulta de filtrado (Backend/Controlador): Filtrar paquetes entregados de la lista de pendientes de guardia.

Historias de Usuario	Tareas técnicas identificadas
HU29 - Publicar un comunicado general	1. Diseñar tabla de anuncios (comunicados): Crear migración con campos y autor. 2. Crear formulario de publicación (Frontend/Blade - Admin): Formulario para redactar avisos. 3. Desarrollar lógica de persistencia (Backend/Controlador): Validar y guardar comunicado. 4. Implementar Colas de Trabajo (Backend): Job para enviar correos masivos de forma asíncrona. 5. Modificar el Dashboard del inquilino (Frontend/Blade): Renderizar últimos comunicados en la página de inicio.
HU31: Enviar consulta directa a la administración	1. Diseñar tabla de mensajería (tickets o consultas): Migración con campos e inquilino_id. 2. Crear formulario de contacto (Frontend/Blade - Inquilino): Formulario para redactar consultas. 3. Desarrollar lógica de persistencia (Backend/Controlador): Guardar consulta asociada al usuario logueado. 4. Maquetar bandeja de entrada (Frontend/Blade - Admin): Bandeja ordenada por fecha de creación. 5. Desarrollar consulta de tickets (Backend/Controlador): Query para listar y filtrar por estado (Pendiente/Resuelto).
HU30 - Consultar documentos del consorcio	1. Diseñar tabla de metadatos (documentos): Crear migración con título, path y fecha. 2. Crear vista de repositorio (Frontend/Blade): Grilla con íconos de los documentos del consorcio. 3. Desarrollar consulta de listado (Backend/Controlador): Recuperar y enviar documentos públicos a la vista. 4. Implementar lógica de entrega de archivos (Backend): Ruta de descarga segura utilizando la clase Storage. 5. Configurar cabeceras de respuesta HTTP: response()- file() para visualización inline en el navegador.

Figura 6: Tabla de identificación de Tareas

2.1.2.5. Estimación de Tareas

Historia de Usuario	Estimación de Tareas	Estimación total
HU1 - Registro del Edificio	1. 2 h 2. 1 h 3. 2 h 4. 3 h 5. 2 h	10 h
HU2 - Configuración de Departamentos/Alquileres	1. 2 h 2. 2 h 3. 2 h	8 h
HU3 - Ingreso de Gastos del Consorcio.	1. 2 h 2. 1 h 3. 2 h	5 h
HU5 - Generar liquidación mensual de expensas	1. 2 h 2. 2 h 3. 2 h 4. 3 h 5. 1 h 6. 2 h	12 h
HU6 - Registro manual de pagos y cobranzas	1. 2 h 2. 2 h 3. 3 h 4. 1 h 5. 2 h	10 h
HU11 - Consultar mi estado de deudas (Inquilino)	1. 2 h 2. 2 h 3. 1 h 4. 3 h 5. 2 h	10 h
HU15 - Consultar gastos de expensa mensual (Inquilino)	1. 1 h 2. 2 h 3. 3 h 4. 3 h	9 h

Historia de Usuario	Estimación de Tareas	Estimación total
HU13 - Pagar de deudas Online (Inquilino)	1. 3 h 2. 2 h 3. 3 h 4. 2 h 5. 2 h 6. 3 h	15 h
HU18 - Solicitar reserva de los espacios comunes	1. 2 h 2. 3 h 3. 4 h 4. 3 h 5. 3 h	15 h
HU20 - Cancelar reserva del espacio común	1. 1 h 2. 3 h 3. 4 h 4. 3 h 5. 2 h 6. 2 h	15 h
HU17 - Ver disponibilidad de los espacios comunes	1. 2 h 2. 3 h 3. 3 h 4. 4 h 5. 3 h	15 h
HU22 - Alta de un nuevo vecino	1. 2 h 2. 2 h 3. 3 h 4. 4 h 5. 5 h	16 h
HU24 - Baja o desvinculación de un inquilino	1. 2 h 2. 3 h 3. 3 h 4. 2 h	10 h

Historia de Usuario	Estimación de Tareas	Estimación total
HU23 - Actualización de datos de contacto	1. 2 h 2. 2 h 3. 3 h 4. 3 h 5. 3 h	13 h
HU26 - Registrar ingreso de visita	1. 2 h 2. 3 h 3. 3 h 4. 4 h 5. 5 h 6. 3 h	20 h
HU27 - Registrar recepción de paquetería	1. 2 h 2. 3 h 3. 4 h 4. 3 h	12 h
HU28 - Confirmar retiro de paquete	1. 2 h 2. 3 h 3. 5 h 4. 4 h	14 h
HU29 - Publicar un comunicado general	1. 2 h 2. 3 h 3. 4 h 4. 5 h 5. 4 h	18 h
HU31 - Enviar consulta directa a la administración	1. 3 h 2. 3 h 3. 3 h 4. 2 h 5. 2 h	13 h
HU30 - Consultar documentos del consorcio	1. 2 h 2. 3 h 3. 4 h 4. 3 h 5. 2 h	14 h

Historia de Usuario	Estimación de Tareas	Estimación total
	1. 2 h 2. 3 h 3. 4 h 4. 3 h 5. 2 h	14 h

Figura 7: Tabla de estimación de tareas.

2.1.2.6. Actualizar el backlog del sprint

Dado que la Liberación 1 está planificada para durar un mes en total, se dividió el trabajo en 2 Sprints de dos semanas de duración cada uno. Para el **Sprint 1**, el Backlog del Sprint se actualizó trasladando las tareas técnicas (previamente identificadas en Trello) correspondientes a las historias fundamentales (Registro de Edificio, Configuración de Unidades, Ingreso de Gastos y Liquidación Mensual). El resto de las historias de la Liberación 1 (como Consultar deudas y Pagar online) quedaron reservadas para la planificación del Sprint 2.

2.1.3. 2.1.3 - Fase - Implementación

2.1.3.1. Crear entregables.

Los desarrolladores proceden a la codificación en el framework Laravel, creando las migraciones de bases de datos, maquetando las vistas (Frontend) y programando los Controladores. A medida que las tareas avanzan, las tarjetas en el tablero Trello se mueven de la columna “Por hacer” a “En progreso” y finalmente a “Terminada”.

2.1.3.2. Realizar el Daily Standup.

El equipo de desarrollo realiza una reunión diaria de máximo 15 minutos donde cada integrante responde tres preguntas clave: ¿Qué hice ayer para ayudar a terminar el Sprint?, ¿Qué haré hoy?, y ¿Existe algún impedimento que me bloquee?

2.1.3.3. Refinar el backlog priorizado del producto.

El Product Owner revisa las historias de usuario de futuras épicas (como el módulo del Personal de Seguridad) para asegurar que estén claras y listas para cuando inicie la planificación de las siguientes iteraciones.

2.1.4. 2.1.4 - Fase - Revisión y Retrospectiva

2.1.4.1. Demostrar y validar el sprint.

Se realiza una demostración del software funcional. Por ejemplo, el equipo muestra cómo el sistema logra tomar los gastos cargados y generar exitosamente la liquidación mensual prorrateada. Si cumple con los Criterios de Aceptación, la historia se marca como “Hecha”.

2.1.4.2. Retrospectiva del sprint.

El equipo evalúa sus procesos internos. Se identifican aspectos positivos (ej. “usar migraciones de Laravel agilizó el trabajo”) y áreas de mejora (ej. “subestimamos el tiempo que tomaba maquetar las vistas”) para aplicar correcciones de cara al Sprint 2.

2.1.5. 2.1.5 - Fase - Liberación

2.1.5.1. Enviar Entregables.

El código fuente se despliega en un servidor de producción. A partir de este momento, la administración del consorcio ya cuenta con un incremento de producto funcional que pueden utilizar en su día a día para gestionar los edificios y las liquidaciones, mientras el equipo de desarrollo comienza a trabajar en la Liberación 2 (Módulo de reservas).

2.1.5.2. Retrospectiva de la liberación.

Tras completar la totalidad de las iteraciones planeadas para el cuatrimestre, se realiza una reunión global final. Se documenta la experiencia general, evaluando si el sistema cumplió con la visión original y analizando el impacto positivo de haber utilizado un marco de trabajo ágil e iterativo frente a las metodologías tradicionales.

2.2. Entrevista

En esta sección se detallan las entrevistas realizadas a los diferentes actores del consorcio para comprender sus necesidades y puntos de vista respecto al funcionamiento del sistema de administración.

Entrevista 1: Administrador del Consorcio

Se utilizó una estructura de embudo para la entrevista hacia el administrador del consorcio.

Preguntas abiertas:

1. ¿Cuáles son los objetivos críticos de su departamento al administrar este edificio?

Administrador: “El objetivo principal es mantener las cuentas al día para poder pagar los sueldos y los servicios sin sobresaltos. Además, necesito asegurar que el edificio esté en buenas condiciones de mantenimiento y, sobre todo, transparentar los gastos para que los vecinos no se quejen todo el tiempo. Básicamente, que la plata alcance y tener paz en el consorcio.”

2. Describa el proceso actual para calcular y distribuir las expensas mensuales.

Administrador: “Hoy es un dolor de cabeza tremendo. A fin de mes, junto todos los tickets de gastos (luz, agua, arreglos, sueldo del encargado), los sumo en una planilla de Excel, divido por los porcentajes de cada departamento según los metros cuadrados, y armo un PDF. Después, tengo que mandarlos uno por uno por mail o imprimir los papelitos para pasarlos por debajo de la puerta. Me lleva días.”

Pregunta de Sondeo:

3. ¿Me lo puede explicar con más detalle para los casos en que hay deudores morosos?

Administrador: “Uf, ese es el mayor problema. En el Excel tengo que ir marcando quién pagó y quién no. Si alguien se atrasa, al mes siguiente tengo que acordarme de sumarle el interés punitivo del 5 % a mano sobre su deuda anterior, más la expensa nueva. A veces se me pasa, o el vecino me dice ‘yo te transferí el día 15’ y tengo que ir a buscar en el resumen del banco línea por línea a ver si es verdad. Es muy poco práctico.”

Preguntas cerradas:

4. Mencione sus dos prioridades principales para mejorar la infraestructura de tecnología del edificio.

Administrador: “Primero, automatizar el cálculo y el envío de las expensas. Segundo, tener un registro automático de los pagos para no tener que cruzar las transferencias bancarias a mano todos los días.”

¿Cuántos subordinados tiene a cargo para el mantenimiento y seguridad que usarán el sistema?

Administrador: “Directamente relacionados con el uso del sistema, tengo a los dos guardias de seguridad que rotan en la entrada y al encargado del edificio. Tres en total.”

Preguntas bipolares:

5. ¿Desea recibir una impresión de su estado de cuenta cada mes?

Administrador: “No, prefiero que quede todo 100 % digital e histórico en el sistema para poder consultarlo desde la computadora cuando quiera, sin acumular más papel.”

6. ¿Está de acuerdo o en desacuerdo con que el actual método manual de reservas de los espacios comunes carece de seguridad?

Administrador: “Totalmente de acuerdo. Hoy con el cuaderno de papel que tiene el guardia, cualquiera tacha un nombre o se anotan dos vecinos en el mismo horario para usar los espacios comunes, y después el problema y las discusiones me explotan a mí.”

Entrevista 2: Propietario/Cliente

Para esta entrevista utilizamos la estructura de pirámide, ya que es la más acorde para el cliente.

Preguntas bipolares:

1. ¿Actualmente recibe el resumen de sus expensas impreso en papel por debajo de su puerta?

Cliente: “Sí, tal cual. Me lo tiran por debajo de la puerta todos los meses.”

Preguntas cerradas:

2. ¿Cuántas veces al año suele reservar los espacios comunes, como el quincho o los espacios comunes?

Cliente: “Calculale unas tres o cuatro veces al año como mucho, más que nada para algún cumpleaños o una cena familiar grande.”

Pregunta de Sondeo:

3. Mencionó que el método para reservar es anotarse en un cuaderno en la guardia, ¿me puede dar un ejemplo de qué ocurre si ese cuaderno se pierde o hay doble reserva?

Cliente: “Uy, es un desastre. Una vez organicé el cumple de mi nene, compré la comida, invité a todos, y cuando bajé al quincho con las cosas había otra familia instalada. Resulta que el guardia del turno mañana me anotó en una hoja, pero el del turno noche no lo vio y le reservó a otro vecino en el mismo horario. Nos terminamos peleando entre nosotros por culpa de la desorganización.”

Preguntas abiertas:

4. ¿Cuáles son los métodos de pago electrónico que preferiría tener integrados en la nueva plataforma?

Cliente: “Para mí sería clave que tenga MercadoPago o algún QR para pagar en el momento desde el celular. Y si es transferencia bancaria, que el sistema te lo tome automático, porque hoy en día tenés que hacer la transferencia y después andar mandándole un mail al administrador con la captura de pantalla para que te lo anote. Es una pérdida de tiempo.”

5. En general, ¿qué opina de la transparencia actual en el detalle de los gastos y cómo cree que el sistema podría mejorar esa comunicación?

Cliente: “La verdad, hoy la transparencia es nula. Te llega esa hoja por debajo de la puerta con un número final gigante y un resumen que no se entiende nada. A veces dice ‘Arreglo de caños: \$200.000’ y vos no tenés idea de qué caño se rompió ni si pidieron presupuesto. Yo creo que si el sistema tuviera una sección tipo ‘Muro’ o de ‘Documentos’ donde suban las facturas de lo que se arregla, o un gráfico para ver en qué se nos va la plata mes a mes, nos daría muchísima más tranquilidad a todos.”

2.3. 2.3 - Especificación de requisitos de Software

2.3.1. 2.3.1 - Introducción

El propósito de este documento es especificar los requerimientos funcionales y no funcionales del sistema CoConsortio. El sistema permitirá administrar consorcios de edificios, facilitando la gestión de departamentos, propietarios, inquilinos, pagos, expensas, reservas y comunicación entre vecinos y administración.

Este documento está dirigido a:

- Administradores
- Empleados administrativos
- Propietarios
- Inquilinos

- Equipo de desarrollo

2.3.1.1. 2.3.1.2 - Alcance

El sistema se denominará “CoConsortio”. El sistema permitirá a los usuarios:

- Gestionar usuarios
- Gestionar departamentos
- Gestionar contratos de alquiler
- Registrar pagos y expensas
- Gestionar reservas de espacios comunes
- Gestionar visitas y paquetería
- Publicar comunicados
- Consultar documentación del consorcio
- Gestionar información de propietarios e inquilinos

El sistema estará disponible mediante una aplicación web accesible desde dispositivos con conexión a internet.

2.3.1.2. 2.3.1.3 - Definiciones, acrónimos y abreviaturas

Término	Definición
ERS	Especificación de Requisitos de Software
RF	Requisito Funcional
RNF	Requisito No Funcional
Consortio	Conjunto de propietarios y departamentos administrados
Expensas	Gastos comunes correspondientes al edificio
Inquilino	Persona que alquila un departamento
Propietario	Persona dueña de una unidad funcional

Figura 8: Tabla de definiciones

2.3.2. 2.3.2 - Descripción General

2.3.2.1. 2.3.2.1 - Perspectiva del producto

CoConsortio será un sistema independiente orientado a la administración de consorcios y alquileres de edificios. El sistema permitirá centralizar la información relacionada con:

- Departamentos,
- Propietarios,
- Inquilinos,
- Pagos,
- Expensas,
- Reservas,
- Reclamos,
- Comunicaciones internas.

Los usuarios accederán mediante una interfaz web conectada a una base de datos centralizada.

2.3.2.2. 2.3.2.2 - Funcionalidad del Producto

El sistema deberá permitir:

- Gestión de usuarios
- Gestión de departamentos
- Gestión de propietarios e inquilinos
- Gestión de pagos y expensas
- Gestión de reservas de espacios comunes
- Gestión de visitas y paquetería
- Gestión de comunicados y documentación
- Gestión de contratos de alquiler

2.3.2.3. 2.3.2.3 - Características de los usuarios

Tipo de usuario	Formación	Experiencia y especialización técnica
Administrador	Secundario/Terciario	Manejo básico de sistemas administrativos
Empleado administrativo	Secundario	Manejo básico de computadoras
Propietario	Variable	Uso básico de aplicaciones web
Inquilino	Variable	Uso básico de aplicaciones web.

Figura 9: Tabla de características de usuarios.

2.3.2.4. 2.3.2.4 - Restricciones

- El sistema deberá funcionar mediante conexión a internet.
- El acceso requerirá autenticación de usuario.
- El sistema deberá operar en navegadores web modernos.
- El sistema utilizará una base de datos relacional.
- El sistema será desarrollado utilizando tecnologías web.
- El desarrollo del proyecto se realizará utilizando metodología Scrum.

2.3.2.5. 2.3.2.5 - Suposiciones y dependencias

- Disponibilidad permanente de conexión a internet.
- Disponibilidad del servidor de alojamiento.
- Correcto funcionamiento de la base de datos.
- Disponibilidad de dispositivos compatibles con navegadores web.
- Los usuarios deberán poseer credenciales válidas para acceder al sistema.

2.3.3. 2.3.3 - Requisitos Específicos

Esta sección describe los requisitos funcionales y no funcionales correspondientes al Sistema de Administración de Consorcios para un Edificio. Los requisitos fueron definidos a partir de las funcionalidades necesarias para la gestión administrativa del consorcio y la interacción entre administradores, propietarios e inquilinos.

2.3.3.1. 2.3.3.1 - Requisitos Funcionales

N° de requisito	Descripción del requisito
RF#01	El sistema debe permitir registrar usuarios ingresando sus datos personales (nombre, DNI, correo electrónico)
RF#02	El sistema debe permitir iniciar sesión
RF#03	El sistema debe permitir cerrar sesión
RF#04	El sistema debe permitir actualizar datos de contacto
RF#05	El sistema debe permitir registrar nuevos vecinos
RF#06	El sistema debe permitir registrar bajas de inquilinos
RF#07	El sistema debe restringir funcionalidades según el tipo de usuario

Figura 10: Tabla de Requisitos: Gestión de usuarios

2.3.3.1.1. Gestión de Usuarios.

N° de requisito	Descripción del requisito
RF#08	El sistema debe permitir registrar pagos de expensas
RF#09	El sistema debe permitir consultar historial de pagos
RF#10	El sistema debe permitir consultar comprobantes de pago
RF#11	El sistema debe permitir subir comprobantes de transferencia
RF#12	El sistema debe permitir confirmar pagos realizados por transferencia
RF#13	El sistema debe generar liquidaciones de expensas
RF#14	El sistema debe permitir consultar estado de deuda
RF#15	El sistema debe permitir enviar notificaciones a deudores

Figura 11: Tabla de requisitos: Gestión de pagos y expensas

2.3.3.1.2. Gestión de pagos y expensas

N° de requisito	Descripción del requisito
RF#16	El sistema debe permitir visualizar disponibilidad de espacios comunes
RF#17	El sistema debe permitir solicitar reservas de espacios comunes
RF#18	El sistema debe permitir cancelar reservas realizadas
RF#19	El sistema debe permitir gestionar solicitudes de reserva
RF#20	El sistema debe permitir consultar historial de reservas

Figura 12: Tabla de requisitos: Gestión de espacios comunes.

2.3.3.1.3. Gestión de espacios comunes.

N° de requisito	Descripción del requisito
RF#21	El sistema debe permitir registrar ingresos visitas
RF#22	El sistema debe permitir registrar recepción de paquetería
RF#23	El sistema debe permitir confirmar retiro de paquetes

Figura 13: Tabla de requisitos: Gestión de visitas y paquetería.

2.3.3.1.4. Gestión de visitas y paquetería.

N° de requisito	Descripción del requisito
RF#24	El sistema debe permitir publicar comunicados generales
RF#25	El sistema debe permitir consultar documentos del consorcio
RF#26	El sistema debe permitir enviar consultas directas a la administración

Figura 14: Tabla de requisitos: Gestión de comunicación y documentos.

2.3.3.1.5. Gestión de comunicación y documentos.

N° de requisito	Descripción del requisito
RF#27	El sistema debe permitir registrar departamentos
RF#28	El sistema debe permitir asociar propietarios a departamentos
RF#29	El sistema debe permitir registrar contratos de alquiler
RF#30	El sistema debe permitir consultar contratos activos
RF#31	El sistema debe permitir la integración con pasarelos de pagos externos.

Figura 15: Tabla de requisitos: Gestión de comunicación y contratos.

2.3.3.1.6. Gestión de departamentos y contratos

2.3.4. 2.3.4 - Requisitos No Funcionales

2.3.4.1. 2.3.4.1 - Requisitos del producto

N° de requisito	Descripción del requisito	Clasificación
RNF#01	El sistema debe ejecutarse correctamente en los navegadores web más utilizados (Chrome, Firefox, Edge) en sus últimas versiones.	Portabilidad
RNF#02	El sistema deberá responder a las solicitudes en un tiempo máximo de 5 segundos	Eficiencia y Rendimiento
RNF#03	El sistema debe permitir que los usuarios naveguen por la interfaz de manera intuitiva, requiriendo un tiempo de capacitación mínimo.	Eficiencia/Usabilidad
RNF#04	El sistema debe garantizar una disponibilidad continua del servicio, con recuperación ante fallas sin pérdida de datos.	Fiabilidad
RNF#05	El sistema debe adaptar su interfaz gráfica correctamente para ser accesible desde dispositivos móviles (Smartphones y Tablets).	Portabilidad
RNF#06	El sistema deberá almacenar la información de manera segura	Fiabilidad
RNF#07	El sistema deberá permitir acceso desde dispositivos móviles	Portabilidad

Figura 16: Tabla de requisitos: Requisitos del producto

2.3.4.2. 2.3.4.2 - Requisitos organizacionales

N° de requisito	Descripción del requisito	Clasificación
RNF#06	El equipo de desarrollo debe utilizar la metodología ágil Scrum para la gestión del proyecto y sus entregables.	Estándares
RNF#07	El sistema debe desarrollarse utilizando tecnologías web (Framework Next.js) y una base de datos relacional.	Implementación
RNF#08	El equipo debe proveer la documentación técnica completa y manuales del proyecto al finalizar el ciclo de desarrollo.	Entrega

Figura 17: Tabla de requisitos: Requisitos organizacionales

2.3.4.3. 2.3.4.3 - Requisitos externos

N° de requisito	Descripción del requisito	Clasificación
RNF#09	El sistema debe requerir una conexión a internet activa para su funcionamiento y comunicación con el servidor de alojamiento.	Interoperabilidad
RNF#10	El sistema debe cumplir con las normativas legales vigentes sobre la protección de datos personales de los consorcistas.	Legislativos/Privacidad
RNF#11	El sistema debe garantizar que el control de acceso, el almacenamiento y los pagos online (MercadoPago) se realicen mediante conexiones cifradas (HTTPS).	Legislativos/Seguridad

Figura 18: Tabla de requisitos: Requisitos Externos

2.4. 2.4 - Análisis de Riesgos

La gestión de riesgos constituye un proceso fundamental para asegurar el éxito del presente proyecto, permitiendo la identificación, análisis y mitigación proactiva de los factores que podrían comprometer sus objetivos. Dada la naturaleza dinámica de nuestro “Sistema de Administración de Consorcios”, el enfoque adoptado se basa en un marco de trabajo continuo que abarca cuatro etapas críticas: la identificación de amenazas potenciales para el negocio, el producto y el cronograma del proyecto; el análisis cualitativo de su probabilidad e impacto; la planificación de estrategias de prevención y contingencia; y, finalmente, la supervisión

constante para ajustar las medidas de control según la evolución del desarrollo.

Este análisis permite al equipo anticipar posibles dificultades, optimizar la asignación de recursos y garantizar la calidad y fiabilidad del software final frente a incertidumbres tanto técnicas como organizacionales.

Riesgo	Tipo	Descripción
Error en cálculos financieros	Producto	Errores en la lógica del prorrateo automático de expensas o en el cálculo de intereses por mora.
Falla en seguridad de datos	Producto	Acceso no autorizado o filtración de datos sensibles de los vecinos y registros del Libro de Visitas.
Falla de integración de pagos	Proyecto (producto)	Problemas técnicos al conectar el sistema con la pasarela de pagos para la generación de recibos.
Nuevos requerimientos del cliente	Negocio	Solicitud de cambios o nuevas funciones (ej. agregar un módulo de votación a mitad del desarrollo).
Baja adopción tecnológica	Negocio	Resistencia de los vecinos (especialmente adultos mayores) a utilizar la aplicación para reservas o pagos.
Falta de experiencia técnica	Proyecto	Dificultad para manejar las herramientas de desarrollo elegidas para el módulo de notificaciones push.
Interfaz poco intuitiva	Producto	Diseño de pantallas confuso que dificulte la carga de tickets de mantenimiento por parte del vecino.
Ausencia de integrantes	Proyecto	Inasistencia de un miembro del equipo por razones de salud o personales durante las fechas de entrega.

Figura 19: Tabla de los riesgos que se pensaron

Tras la identificación, se procede a la valoración cualitativa de cada riesgo (Probabilidad y Seriedad). Finalmente, se define el plan de gestión y la estrategia de mitigación para cada uno.

Riesgo	Probabilidad	Seriedad
Falla de seguridad de datos	Baja	Alta
Error en cálculos financieros	Media	Alta
Ausencia de integrantes	Media	Alta
Nuevos requerimientos del cliente	Alta	Media
Baja adopción tecnológica	Media	Media
Falla de integración de pagos	Alta	Media
Falta de experiencia técnica	Media	Baja
Interfaz poco intuitiva	Baja	Baja

Figura 20: Tabla con las probabilidades y seriedad de los riesgos

A continuación, se detalla la estrategia de gestión y el plan de acción específico para la prevención o mitigación de cada riesgo.

Riesgo	Estrategia	Plan de Gestión (Detalle)
Falla de seguridad de datos	Prevención	Implementar cifrado de datos sensibles y auditorías de acceso desde la fase de diseño.
Error en cálculos financieros	Prevención	Realizar pruebas unitarias intensivas sobre la lógica del prorrateo y validaciones cruzadas.
Ausencia de integrantes	Minimización	Documentar todo el código y procesos para que cualquier miembro pueda retomar tareas de otro.
Nuevos requerimientos del cliente	Prevención	Establecer un alcance claro (Scope) y utilizar un mecanismo formal de gestión de cambios.
Baja adopción tecnológica	Minimización	Crear tutoriales breves y realizar una prueba de usabilidad con un grupo control de adultos mayores.
Falla de integración de pagos	Contingencia	Tener un plan de respaldo manual para la generación de recibos si la API de pago falla.
Falta de experiencia técnica	Prevención	Capacitación temprana mediante workshops técnicos o tutoriales sobre notificaciones push.
Interfaz poco intuitiva	Prevención	Realizar prototipado rápido y validación con usuarios reales antes de la codificación final.

Figura 21: Estrategias para cada riesgo

2.4.1. 2.5 - Planificación del Proyecto

La planificación del Sistema de Administración de Consorcio se rige bajo la metodología ágil Scrum, estructurando el trabajo mediante iteraciones denominadas Sprints. Se ha definido un esquema de entregas basado en el valor de negocio, segmentado en tres grandes liberaciones con un Time-boxing de dos semanas por cada Sprint.

2.4.1.1. 2.5.1 - Liberación 1: Gestión Financiera Base

Esta liberación constituye el núcleo empírico del sistema.

- **Sprint 1:** Comprende la configuración estructural y el motor contable. **Historias**

comprometidas: HU1 (Registro del Edificio), HU2 (Configuración de Departamentos), HU3 (Ingreso de Gastos), HU5 (Generar liquidación), y HU6 (Registro manual de pagos). **Esfuerzo total estimado:** 45 h

- **Sprint 2:** Enfocado en la visibilidad y métodos de pago para el usuario final. **Historias comprometidas:** HU11 (Consultar deudas), HU15 (Consultar gastos), y HU13 (Pagar deudas Online). **Esfuerzo total estimado:** 34 h

2.4.1.2. 2.5.2 - Liberación 2: Gestión de Espacios y Usuarios

Expande el valor entregado incorporando la operatividad diaria del residente.

- **Sprint 1:** Centrado en el módulo de reservas. **Historias comprometidas:** HU17 (Ver disponibilidad), HU18 (Solicitar reserva), y HU20 (Cancelar reserva). **Esfuerzo total estimado:** 45 h
- **Sprint 2:** Dedicado a la gestión del padrón de vecinos. **Historias comprometidas:** HU22 (Alta de vecino), HU23 (Actualización de datos), y HU24 (Baja/desvinculación). **Esfuerzo total estimado:** 39 h

2.4.1.3. 2.5.3 - Liberación 3: Seguridad y Transparencia

Abarca las funcionalidades operativas para el control físico y la comunicación institucional.

- **Sprint 1:** Implementación del libro digital de guardia. **Historias comprometidas:** HU26 (Ingreso de visita), HU27 (Recepción de paquetería), y HU28 (Confirmar retiro). **Esfuerzo total estimado:** 46 h
- **Sprint 2:** Módulo de comunicación y repositorio documental. **Historias comprometidas:** HU29 (Publicar comunicado), HU30 (Consultar documentos), y HU31 (Enviar consulta). **Esfuerzo total estimado:** 45 h

2.4.1.4. 2.5.4 - Actividades y Ceremonias Ágiles

Durante el desarrollo de estos Sprints, el equipo ejecutará las siguientes ceremonias:

- **Daily Standup:** Reuniones diarias para sincronizar tareas respondiendo qué se hizo, qué se hará y qué impedimentos existen.
- **Revisión del Sprint:** Demostración del software funcional al finalizar la iteración para validar los criterios de aceptación.
- **Retrospectiva del Sprint:** Evaluación de procesos internos para identificar áreas de mejora y optimizar el flujo de trabajo en la siguiente iteración.

3. Herramientas y/o lenguajes de programación

Dado que la presente etapa del proyecto “CoConsorcio” se centró en el relevamiento, la especificación de requisitos y la simulación de la gestión bajo un marco de trabajo ágil, no se emplearon lenguajes de programación ni entornos de desarrollo de código. En su lugar, el equipo hizo uso exclusivo de herramientas de software orientadas a la administración de proyectos, la redacción colaborativa y la comunicación.

A continuación, se describen brevemente las herramientas utilizadas:

- **Google Docs:** Procesador de textos basado en la nube utilizado para la redacción colaborativa en tiempo real. Fue la herramienta principal para estructurar, revisar y unificar toda la documentación académica del proyecto, incluyendo la Especificación de Requisitos de Software (ERS) y la transcripción de entrevistas.
- **Trello:** Software de administración de proyectos basado en la metodología de tableros visuales Kanban. Se utilizó para simular y gestionar el ciclo de vida del marco de trabajo Scrum, permitiendo al equipo administrar el *Product Backlog*, planificar los Sprints y realizar el seguimiento del estado de las Historias de Usuario.
- **Gemini (Inteligencia Artificial):** Asistente basado en modelos de lenguaje de gran tamaño (LLM) empleado como herramienta de apoyo metodológico. Se utilizó para estructurar ideas, refinar la redacción técnica de los requerimientos bajo el estándar IEEE y generar escenarios de simulación (datos de prueba para entrevistas e historias de usuario).
- **Discord:** Plataforma de comunicación mediante canales de voz y texto. Se empleó para llevar a cabo las reuniones sincrónicas del equipo, especialmente para coordinar las tareas, simular las reuniones ágiles (*Daily Standups*) y compartir pantalla durante la revisión de los documentos.
- **WhatsApp:** Aplicación de mensajería instantánea utilizada como canal secundario para la comunicación asincrónica y rápida del equipo. Facilitó la organización de horarios, avisos urgentes y el intercambio de enlaces directos a los recursos del proyecto.